

NumPy Tutorial 4: Indexing and Slicing — Accessing Data from Arrays

This chapter covers indexing and slicing in NumPy. After completing this tutorial, students will understand how to access specific elements, rows, columns, and portions of arrays.

1. Introduction

In the previous tutorial, we learned how to perform operations on arrays. In this tutorial, we will learn how to access specific data from arrays.

In real world Data Science, we often need to extract a specific row, column, or range of values from a dataset. NumPy makes this very easy using indexing and slicing.

2. What is Indexing

Indexing means accessing a specific element from an array using its position number.

In NumPy, indexing starts from **0** — meaning the first element is at position 0.

3. Indexing in 1D Arrays

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print(arr[0])
print(arr[2])
print(arr[4])
```

Output:

```
10
30
50
```

Negative Indexing

We can also use negative numbers to access elements from the end.

```
print(arr[-1])  
print(arr[-2])
```

Output:

```
50  
40
```

- -1 gives the last element.
- -2 gives the second last element.

4. Indexing in 2D Arrays

For 2D arrays, we use two index numbers — one for row and one for column.

Format: `arr[row, column]`

```
import numpy as np  
  
arr = np.array([[1, 2, 3],  
                [4, 5, 6],  
                [7, 8, 9]])  
  
print(arr[0, 0])  
print(arr[1, 2])  
print(arr[2, 1])
```

Output:

```
1  
6  
8
```

In this example:

- `arr[0, 0]` — row 0, column 0 — gives 1.
- `arr[1, 2]` — row 1, column 2 — gives 6.
- `arr[2, 1]` — row 2, column 1 — gives 8.

5. What is Slicing

Slicing means selecting a range of elements from an array — not just one element.

Format: `arr[start:stop:step]`

- **start** — where to begin (included)
- **stop** — where to end (not included)
- **step** — how many steps to jump

6. Slicing in 1D Arrays

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70])

print(arr[1:5])
print(arr[0:6:2])
print(arr[:3])
print(arr[4:])
print(arr[::-1])
```

Output:

```
[20 30 40 50]
[10 30 50]
[10 20 30]
[50 60 70]
[70 60 50 40 30 20 10]
```

In this example:

- `arr[1:5]` — from index 1 to 4.
- `arr[0:6:2]` — from index 0 to 5, jumping 2 steps.
- `arr[:3]` — from start to index 2.
- `arr[4:]` — from index 4 to end.
- `arr[::-1]` — reverse the array.

7. Slicing in 2D Arrays

For 2D arrays, we slice both rows and columns separately.

Format: `arr[row_start:row_stop, col_start:col_stop]`

```
import numpy as np

arr = np.array([[1, 2, 3, 4],
                [5, 6, 7, 8],
                [9, 10, 11, 12],
                [13, 14, 15, 16]])

print(arr[0:2, 0:2])
print(arr[1:3, 2:4])
print(arr[:, 1])
print(arr[2, :])
```

Output:

```
[[1 2]
 [5 6]]
[[ 7  8]
 [11 12]]
[ 2  6 10 14]
[ 9 10 11 12]
```

In this example:

- `arr[0:2, 0:2]` — first 2 rows, first 2 columns.
- `arr[1:3, 2:4]` — rows 1 and 2, columns 2 and 3.
- `arr[:, 1]` — all rows, column 1 only.
- `arr[2, :]` — row 2 only, all columns.

8. Selecting Entire Row or Column

To select an entire row:

```
print(arr[1])
```

Output:

```
[5 6 7 8]
```

To select an entire column:

```
print(arr[:, 2])
```

Output:

```
[ 3  7 11 15]
```

The colon : means "select all" — it is used very frequently in Data Science.

9. Modifying Array Values Using Indexing

We can also change values in an array using indexing.

```
arr = np.array([10, 20, 30, 40, 50])
arr[2] = 99
print(arr)
```

Output:

```
[10 20 99 40 50]
```

For 2D arrays:

```
arr[1, 2] = 100
print(arr)
```

This changes the value at row 1, column 2 to 100.

10. Fancy Indexing

Fancy indexing means selecting multiple specific elements at once using a list of indices.

```
arr = np.array([10, 20, 30, 40, 50, 60])

indices = [0, 2, 4]
print(arr[indices])
```

Output:

```
[10 30 50]
```

This selects elements at positions 0, 2, and 4 — very useful when you need specific data points.

11. Conclusion of Tutorial 4

In this tutorial, students have learned:

1. What indexing is and how it works in 1D and 2D arrays.
2. How to use negative indexing to access elements from the end.
3. What slicing is and how to use start, stop, and step.
4. How to slice rows and columns in 2D arrays.
5. How to select entire rows and columns using colon.
6. How to modify array values using indexing.
7. What fancy indexing is and how to use it.

In the next tutorial, we will learn how to reshape and manipulate the structure of arrays.

This completes NumPy Tutorial 4.